

Quizora

Backend API — Technical Documentation

Phase 1 (MVP) · Spring Boot 3.5 · PostgreSQL

Every file, service and endpoint explained

Generated: 2 October 2026

Location: `C:\Users\agbes\Desktop\Quizora`

Table of Contents

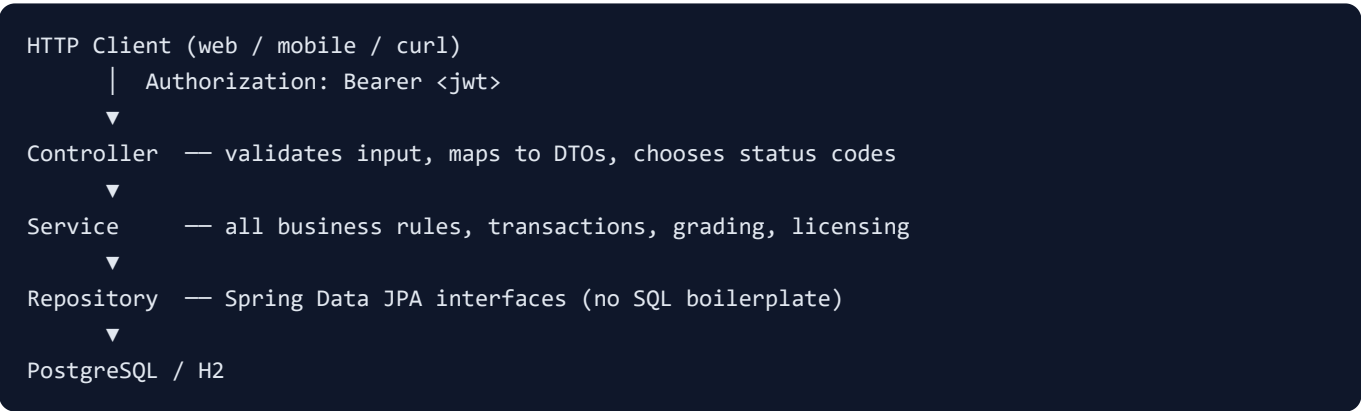
- 1. Overview & Tech Stack
- 2. Project Structure
- 3. Root & Configuration Files
- 4. `config/` — Security & Seed Data
- 5. `security/` — JWT Authentication
- 6. `domain/` — Entities & Enums
- 7. `repository/` — Data Access
- 8. `dto/` — Request/Response Contracts
- 9. `exception/` — Error Handling
- 10. `service/` — Business Logic (11 services)
- 11. `controller/` — REST Endpoints
- 12. Tests
- 13. Demo Accounts & Seeded Data
- 14. API Reference Summary

1. Overview & Tech Stack

Quizora is a past-questions learning platform where students prepare for external examinations (BECE now, WASSCE later) by solving past questions with timers, analytics and institutional monitoring. This backend implements **Phase 1 (MVP)**: authentication, courses, a question bank, a quiz engine, subscriptions, institutional licensing and reporting.

Layer	Choice	Notes
Language	Java 17+ (runs on JDK 17–25)	Compiled to target 17
Framework	Spring Boot 3.5.16	Web, Data JPA, Security, Validation
Database	PostgreSQL (default) · H2 in-memory	H2 powers the <code>demo</code> profile and all tests
Auth	JWT — HS256 (jjwt 0.12.7)	Stateless, role-based access control
Reports	CSV (built-in) + PDF (OpenPDF 1.3.43)	Institutional export downloads
Build	Maven wrapper (<code>./mvnw</code>)	No Maven install needed

Architecture (classic 3-layer)



Cross-cutting: JWT filter (auth), GlobalExceptionHandler (uniform JSON errors), DataSeeder (demo data), @Scheduled sweep (subscription expiry).

2. Project Structure

```

Quizora/
├─ pom.xml                # dependencies & build config
├─ mvnw / mvnw.cmd / .mvn/ # Maven wrapper (no install needed)
├─ docker-compose.yml     # PostgreSQL 16 for local dev
├─ README.md              # quick start + API reference
├─ .gitignore
├─ docs/                  # this document
├─ src/
│   └─ main/
│       ├── java/com/quizora/backend/
│       │   ├── QuizoraApplication.java    # entry point
│       │   ├── config/ (2 files)         # SecurityConfig, DataSeeder
│       │   ├── security/ (5 files)        # JWT service/filter/principal
│       │   ├── domain/ (17 files)         # 9 entities + 8 enums
│       │   ├── repository/ (9 files)      # JPA repositories
│       │   ├── dto/ (8 files)             # request/response records
│       │   ├── exception/ (6 files)       # error handling
│       │   ├── service/ (11 files)        # business logic
│       │   └─ controller/ (8 files)       # REST endpoints
│       └─ resources/
│           ├── application.yml             # PostgreSQL (default)
│           └─ application-demo.yml        # H2 in-memory (demo)
└─ test/java/...                          # 2 test classes (4 tests)

```

78 project files total. Conventions: one public class per file; DTOs grouped as nested records inside a final class per feature area; services own all transactions; controllers never touch repositories directly.

3. Root & Configuration Files

pom.xml BUILD

Maven build descriptor. Inherits the **Spring Boot 3.5.16** parent (dependency versions managed centrally), targets **Java 17**, and declares: spring-boot-starter-web/data-jpa/security/validation, **jjwt 0.12.7** (JWT), **OpenPDF 1.3.43** (report export), **PostgreSQL** driver, **H2** (demo/tests), Lombok-free (plain getters/setters), plus spring-boot-starter-test.

mvnw / mvnw.cmd / .mvn/ BUILD

Maven Wrapper — pins Maven 3.9.x and downloads it on first use, so any machine with a JDK can build with `./mvnw test` or `./mvnw spring-boot:run` without installing Maven.

docker-compose.yml INFRA

One-command **PostgreSQL 16** for production-like local runs: exposes port 5432, database `quizora`, user/password `postgres/postgres`, a named volume for data persistence. Usage: `docker compose up -d` then `./mvnw spring-boot:run`.

src/main/resources/application.yml CONFIG

Default profile configuration:

- **Datasource** — PostgreSQL via env vars `DB_URL` / `DB_USER` / `DB_PASSWORD` (sensible localhost defaults).
- **JPA** — `ddl-auto: update` (schema auto-sync for development; swap to Flyway/Liquibase before production), `open-in-view: false`, UTC timezone, ISO date serialization.
- **quizora.security.jwt** — HS256 secret from `JWT_SECRET` (dev fallback ≥ 32 chars), token lifetime 24 h.
- **quizora.cors.allowed-origins** — from `CORS_ORIGINS`, default `*`.
- **quizora.seed.enabled=true** — toggles DataSeeder; **quizora.subscription.trial-days=7** — free trial length.

[src/main/resources/application-demo.yml](#)

CONFIG

The zero-setup `demo` profile: swaps the datasource to **in-memory H2** in PostgreSQL compatibility mode (`MODE=PostgreSQL`), `create-drop` schema, enables the H2 web console. Every run starts fresh and re-seeds demo data — ideal for local exploration and for the integration tests.

[README.md](#)

DOCS

Team-facing guide: tech stack, quick start (demo vs PostgreSQL), demo accounts, implemented features, full API reference tables, project structure and the phase roadmap.

[QuizoraApplication.java](#)

ENTRY

Spring Boot entry point (`@SpringBootApplication`). Also carries `@EnableScheduling` , which activates the nightly cron job that expires ended subscriptions (see `SubscriptionService`).

4. `config/` — Security & Seed Data

SecurityConfig.java SECURITY

The heart of the security setup (`@Configuration @EnableWebSecurity @EnableMethodSecurity`). Three beans:

`filterChain(HttpSecurity)`

- Stateless sessions (no cookies — JWT only), CSRF disabled (not needed for token auth).
- **Route rules:** `/api/auth/**`, `/actuator/health`, `/h2-console/**` and all OPTIONS preflights are **public**; `/api/admin/**` requires role **ADMIN**; **everything else requires authentication**.
- Inserts `JwtAuthenticationFilter` before Spring's username/password filter.
- Custom JSON 401/403 bodies (never HTML error pages), frame-options `sameOrigin` so the H2 console works.

`corsConfigurationSource()`

Origins from `quizora.cors.allowed-origins` (comma-separated), all standard methods/headers allowed. `allowCredentials=false` deliberately — JWT rides in the `Authorization` header, and Spring rejects wildcard origins combined with credentials.

`passwordEncoder()`

BCrypt — all stored passwords are salted hashes; plaintext never persisted.

DataSeeder.java SEED 239 lines

A `CommandLineRunner` that runs on startup when `quizora.seed.enabled=true` (and only if the data is absent — safe to re-run). Four phases:

1. **seedUsers()** — platform admin, demo student (yearly subscription pre-activated), institution admin and an institution student.
2. **seedQuotes()** — 12 motivational quotes that power the daily dashboard quote.
3. **seedCoursesAndQuestions()** — 6 JHS subjects (Integrated Science, Maths, English, Social Studies, Creative Arts, Ghanaian Language) with **44 BECE-style objective questions** across the 2023 & 2024 papers — each with 4 options, a correct answer and an explanation.
4. **seedInstitution()** — "Demo JHS Accra" plus licence code `DEMO-CODE-1234` (20 seats, valid 2026-01-01 → 2027-12-31).

5. security/ — JWT Authentication

How a request is authenticated, step by step:

1. Client logs in → AuthService verifies BCrypt hash
2. JwtService.generateToken(user) signs HS256 JWT (id + email + role, 24 h)
3. Client sends Authorization: Bearer <token> on every request
4. JwtAuthenticationFilter parses & verifies the token
5. QuizoraUserDetails placed in SecurityContext → @CurrentUser injects User

JwtService.java SECURITY

- generateToken(User) — signs a JWT with subject=email, claims uid & role, 24 h expiry (configurable expiration-ms).
- parse(String) — verifies signature + expiry and returns the claims (throws on invalid tokens).
- getExpirationMs() — surfaced to clients so they can show token lifetime / schedule refresh.

JwtAuthenticationFilter.java SECURITY

A OncePerRequestFilter that runs before authorization. Reads the Authorization: Bearer ... header, parses the token, loads the user via QuizoraUserDetailsService, and — only if the user isn't already in the SecurityContext — sets the authentication. Malformed/expired tokens are silently ignored so the request falls through to the 401 entry point.

QuizoraUserDetails.java SECURITY

Implements Spring's UserDetails : wraps the DB user as id, role, email (username), password hash. Authorities are derived from the role (ROLE_STUDENT / ROLE_INSTITUTION_ADMIN / ROLE_ADMIN), which is what makes hasRole("ADMIN") work.

QuizoraUserDetailsService.java SECURITY

Bridges Spring Security to the UserRepository : loadUserByUsername(email) (used by the filter; case-insensitive lookup with join-fetch of the institution to avoid lazy-init issues) and loadByEmail for internal use. Throws UsernameNotFoundException when unknown.

CurrentUser.java

SECURITY

A small helper injected into controllers/services: `get()` returns the authenticated `User` or throws 401/403; `find()` returns `Optional`. Backed by `UserRepository.findWithInstitution` (join fetch) so lazy relationships are already loaded.

6. `domain/` — Entities & Enums (17 files)

Entities (9)

User.java ENTITY

Account record: `id`, `email` (unique, case-insensitive), `passwordHash` (BCrypt), `fullName`, `role` (enum), optional `institution` and the `licenseCode` used to join one. Every quiz, subscription and seat belongs to a user.

Institution.java ENTITY

A school/organisation: `name`, `contactName`, `contactEmail`, `phone`. Owns licence codes and its students; powers the institutional portal and reports.

LicenseCode.java ENTITY

Bulk-seat licence: unique `code` (format `QZRA-XXXX-XXXX`), parent `institution`, `totalSeats` / `usedSeats`, validity window `validFrom` → `validUntil`, `status` (LicenseStatus). A student "consumes" one seat when registering with the code.

Subscription.java ENTITY

Access grant for one user: `plan` (MONTHLY/YEARLY/TRIAL — PlanType), `startDate`, `endDate`, `paymentRef` (mock `MOCK-PAY-...` today), `status` (SubscriptionStatus). Expiry is checked lazily on read and swept nightly.

Course.java ENTITY

A subject/paper: `name` ("Integrated Science"), unique `code` ("SCIENCE"), `description`, `level` (JHS/SHS — Level enum, SHS reserved for Phase 3), `createdAt`. Parent of all its questions.

Question.java ENTITY

A past question: `course` , `examYear` (column is `exam_year` because `year` is a reserved word in H2/SQL), `section` (OBJECTIVE now / SUBJECTIVE in Phase 2), `questionText` , four options `optionA-D` , `correctOption` (AnswerOption enum), `explanation` shown after submission, `marks` (default 1), `createdAt` . Helper `optionAt(answer)` resolves an option letter to its text. Indexed on (course_id, exam_year).

QuizAttempt.java ENTITY

One sitting of a paper: `user` , `course` , `exam_year` , `timed` flag + `timeLimitSeconds` (server-authoritative clock), `status` (AttemptStatus: IN_PROGRESS → SUBMITTED/EXPIRED), `startedAt` / `submittedAt` , results `totalQuestions` , `answeredCount` , `correctCount` , `scorePercent` , `durationSeconds` . Domain logic lives on the entity: `expiresAt()` computes the deadline and `isExpiredNow()` tells the service when to auto-grade.

AttemptAnswer.java ENTITY

One saved answer inside an attempt: link to `attempt` + `question` , the `selectedOption` , whether it was `correct` , and `timeSpentSeconds` . Rows are written as the student answers (so an expiry still captures partial work), then graded results are computed on submit.

MotivationalQuote.java ENTITY

Simple `text` + `author` row; 12 seeded quotes rotated deterministically once per day for the dashboard.

Enums (8)

File	Values	Purpose
Role	STUDENT, INSTITUTION_ADMIN, ADMIN	Access level; drives JWT authorities & route rules
Level	JHS, SHS	Course level (SHS reserved for WASSCE phase)
QuestionSection	OBJECTIVE, SUBJECTIVE	Phase 1 uses OBJECTIVE only
AnswerOption	A, B, C, D	MCQ option letter
AttemptStatus	IN_PROGRESS, SUBMITTED, EXPIRED	Quiz attempt lifecycle
PlanType	TRIAL, MONTHLY, YEARLY	Subscription plan
SubscriptionStatus	ACTIVE, EXPIRED, CANCELLED...	Subscription state
LicenseStatus	ACTIVE, EXHAUSTED, EXPIRED...	Licence state (seat count / dates)

7. repository/ — Data Access (9 files)

All repositories extend `JpaRepository<Entity, Long>`, inheriting CRUD + pagination. Derived queries are declared as method signatures — Spring Data implements them automatically. Custom queries use `@Query` / `@Modifying`.

Repository	Custom methods
UserRepository	<code>existsByEmailIgnoreCase</code> (registration check), <code>findWithInstitution</code> (join fetch — avoids LazyInit), <code>countByRole</code> , <code>countByInstitutionId</code> , <code>countByInstitutionIdAndRole</code> (student-only counts for reports)
CourseRepository	<code>existsByCodeIgnoreCase</code> , plus year/aggregation queries feeding the courses & years endpoints
QuestionRepository	<code>countByCourseIdAndYear</code> , <code>countByCourseId</code> (question counts shown on courses), selection of a paper's questions for a quiz
QuizAttemptRepository	Paged history per user, recent attempts, aggregate stats for dashboard/progress/institution reports
AttemptAnswerRepository	<code>countByAttemptId</code> , <code>countByAttemptIdAndCorrectTrue</code> (answer-vs-total checks)
SubscriptionRepository	<code>findByUserId</code> and the <code>expireEnded()</code> <code>@Modifying @Query</code> used by the nightly sweep to flip ended rows to EXPIRED in one statement
InstitutionRepository	Lookups by contact/name for uniqueness
LicenseCodeRepository	<code>findByCode</code> (registration validation), seat-used sums
MotivationalQuoteRepository	<code>count()</code> + ordered list for deterministic daily rotation; <code>findAll</code> fallback

8. `dto/` — Request/Response Contracts (8 files)

DTOs are Java **records** grouped in final classes (one per feature), so an API contract is immutable and self-documenting. Request records carry `jakarta.validation` annotations (`@NotBlank` , `@Email` , `@Size` , `@Min` ...) enforced automatically by `@Valid` in the controllers. **Correct answers never appear in quiz-facing DTOs** — only in the post-submit review and admin listings.

File	Key records
AuthDtos	<code>RegisterRequest</code> , <code>InstitutionalRegisterRequest</code> , <code>LoginRequest</code> , <code>JoinCodeRequest</code> , <code>SubscribeRequest</code> , <code>UserDto</code> , <code>SubscriptionDto</code> , <code>AuthResponse</code> (JWT + profile)
CourseDtos	<code>CourseResponse</code> (with <code>questionCount</code> + available <code>years</code>) , <code>YearResponse</code>
QuizDtos	<code>StartRequest/Response</code> , <code>QuestionResponse</code> (one question, no correct answer, includes <code>remainingSeconds</code>) , <code>AnswerRequest/Response</code> , <code>ReviewItem</code> (correct + explanation), <code>ResultResponse</code> , <code>AttemptSummary</code>
DashboardDtos	<code>DashboardResponse</code> (greeting, quote, analytics, subscription), <code>QuoteDto</code> , <code>AnalyticsDto</code> (accuracy, average score, streak...)
ProgressDtos	<code>ProgressResponse</code> = <code>TotalsDto</code> + <code>CourseProgressDto[]</code> + <code>TrendPointDto[]</code> (daily graph) + recent attempts
InstitutionDtos	<code>CreateInstitutionRequest</code> , <code>CreateLicenseRequest</code> , <code>LicenseDto</code> (seat usage), <code>InstitutionResponse</code> , <code>StudentDto</code> (per-student snapshot), <code>ReportSummaryResponse</code>
AdminDtos	<code>CreateCourseRequest</code> , <code>QuestionRequest</code> , <code>BulkQuestionRequest/Item</code> (full-paper upload), <code>AdminQuestionResponse</code> (includes correct answer), <code>UserResponse</code> , <code>ActivateSubscriptionRequest</code> , <code>AdminStatsResponse</code> , <code>MessageResponse</code>
Mappers	Static entity→DTO converters (e.g. <code>attemptSummary(QuizAttempt)</code>) shared by services

9. exception/ — Error Handling (6 files)

Services throw typed exceptions; one `@RestControllerAdvice` turns them into a uniform JSON body:

```
{ "status": 404, "error": "Not Found",  
  "message": "Course not found: 99", "path": "/api/courses/99",  
  "timestamp": "2026-10-02T10:30:00" }
```

ApiException.java BASE

Base runtime exception carrying an HTTP `HttpStatus` + message — the common parent for all typed errors below.

NotFoundException.java 404

Resource doesn't exist (e.g. "Question not found: 12").

BadRequestException.java 400

Semantically invalid input (e.g. answering a question that isn't in the attempt).

ForbiddenException.java 403

Authenticated but not allowed (e.g. an institution admin touching another school's data).

ConflictException.java 409

Duplicate resource (e.g. email already registered, duplicate course code).

GlobalExceptionHandler.java ADVICE

Maps every failure path to the JSON shape above: `ApiException` → its status;
`MethodArgumentNotValidException` → 400 with each field message joined ("email: must be a valid email; ..."); `BadCredentialsException` → 401 "Invalid email or password" (never reveals which half was wrong); `AccessDeniedException` → 403; any other `Exception` → 500 with a safe message.

10. `service/` — Business Logic (11 files)

Services hold **all** business rules and are the transaction boundary (`@Transactional` on writes, `@Transactional(readOnly = true)` on reads — which also fixes lazy-loading and enables Hibernate read optimizations). Controllers stay thin; repositories never leak upward.

`AuthService.java` SERVICE

Everything about accounts and sign-in.

- `register()` — individual sign-up: validates uniqueness (case-insensitive email), BCrypt-hashes the password, creates the user, and grants a **7-day free trial** via `SubscriptionService.createTrial()` ; returns JWT + profile.
- `registerInstitutional()` — same flow but requires a valid licence code (`requireUsableLicense` : exists, active, seats left, within validity window), links the institution, and **consumes one seat**; grants an institutional (licence-backed) subscription.
- `login()` — verifies credentials (401 on bad password), returns token response.
- `joinCode()` — lets an existing student link their account to an institution later with a licence code.
- `toUserDto()` — builds the profile DTO (incl. subscription with `daysRemaining`).

`SubscriptionService.java` SERVICE

Access control & the subscription lifecycle — the future Paystack/MoMo integration point.

- `requireActive(user)` — gate called before starting a quiz; throws 403 when there is no active subscription.
- `createTrial()` — 7 days from today (`quizora.subscription.trial-days`).
- `createInstitutional()` — subscription spanning the licence's validity window.
- `activate(user, plan, days, paymentRef)` — activates MONTHLY (30 d) / YEARLY (365 d) with a **mock payment ref** (`MOCK-PAY-...`); if an active sub exists, the new period **extends from the current end date** (never shortens what the user already paid for).
- `lazyExpire()` — flips a stale row to EXPIRED whenever it's read.
- `expireEndedSubscriptions()` — `@Scheduled(cron "0 0 3 * * *")` nightly sweep at 03:00 marking all ended subscriptions EXPIRED.

Swapping in real payments later: keep this service's lifecycle; replace the mock `activate()` call with a Paystack/MoMo webhook that invokes the same method with the real transaction reference.

CourseService.java SERVICE

Read side of the courses module: `list()` (every subject with its `questionCount` and available exam `years`), `get(id)` detail, `years(id)` — per-year question counts so the UI can enable/disable year buttons, and `requireCourse()` used by quiz/admin flows (404 if missing).

QuizService.java SERVICE largest service — the quiz engine

Implements the exam experience end-to-end with **server-authoritative timing and grading** (a client can never see or fake the correct answer or the clock).

- `start()` — checks subscription, loads the paper's questions for (course, year), creates a `QuizAttempt` (optionally timed) and returns metadata: attempt id, total questions, `expiresAt`.
- `question(attemptId, index)` — serves **one question at a time**: text + four options + the student's saved answer + `remainingSeconds`. Ownership check (`requireOwned`) and in-progress check; **never includes correctOption**. If the clock has run out it auto-submits first.
- `answer()` — saves/overwrites the selected option with time spent (upsert by question).
- `submit()` — idempotent: grades all answers server-side, computes counts/score/duration, marks status SUBMITTED (or EXPIRED if the timer elapsed), and returns the full result.
- `result()` — the graded paper: totals, score percent, and a **review list** with the correct option + explanation per question.
- `history()` — paginated past attempts.
- Privates: `requireOwned`, `requireInProgress`, `finalizeAttempt` (grading), `buildResult`, `remainingSeconds`.

DashboardService.java SERVICE

Powers the student home screen in one call (`dashboard(user)`): time-of-day **greeting**, profile, the **daily quote**, **analytics** (courses attempted, questions attempted, accuracy %, average score, total time, study **streak** — consecutive days with at least one attempt, computed by `streakDays()`), and the current subscription summary.

ProgressService.java SERVICE

Analytics behind the Progress page (all read-only transactions):

- `progress()` — overall `TotalsDto` (attempts, questions, accuracy, time spent), **per-course breakdown**, a **daily trend** (`buildTrend()` : attempts + average score per day for the graph), and recent attempts.
- `attemptHistory()` — paginated attempt list.

QuoteService.java SERVICE

`todayQuote()` — deterministic daily rotation: $\text{index} = \text{days-since-epoch} \bmod \text{quote count}$, so every client sees the same quote for a whole 24 h window with no per-request randomness. Falls back to a built-in quote if the table is empty. `validUntil` tells clients when it changes.

InstitutionService.java SERVICE

The institutional (B2B) side:

- `create()` — platform admin creates an institution + its admin account in one step.
- `get()` — overview with licence codes and seat usage (after `requireInstitutionAccess()` : only that institution's admin or a platform admin).
- `createLicense()` — generates a unique `QZRA-XXXX-XXXX` code (`generateCode()` / `randomBlock()` with collision retry) for N seats within a validity window.
- `students()` — per-student snapshots (attempts, accuracy, average score, last active, subscription status) built by `buildStudents()` , which **filters to Role.STUDENT** so the institution's own admin is excluded from their report.
- `summary()` — reporting-dashboard aggregates: totals, 30-day active students, accuracy, seats used/total.

ReportService.java SERVICE

Builds downloadable institution reports as `byte[]` :

- `toCsv()` — header row + one row per student (name, email, attempts, questions, accuracy, average score, last active, subscription), RFC-4180 escaping for commas/quotes.
- `toPdf()` — OpenPDF document with a styled header (institution name, date) and a table of the same columns using `cell()` helpers.

Both enforce `requireInstitutionAccess()` first — a school can only export its own data.

AdminService.java SERVICE

Back-office operations for the platform admin:

- `stats()` — dashboard counts: users, students, institution admins, institutions, courses, questions, attempts.
- `createCourse()` / `createQuestion()` — single additions with uniqueness validation (409 on duplicate course code).
- `createQuestionsBulk()` — **bulk paper upload**: a whole year's questions in one JSON payload; returns how many were created.
- `listQuestions(courseId?, year?)` — optional filters; **includes correct answers** (admin-only endpoint).
- `deleteQuestion()` , `listUsers(role?)` .
- `activateSubscription()` — manual activation/extension for a user (support tool, same extension semantics as self-service).

11. `controller/` — REST Endpoints (8 files)

Thin layer: validate with `@Valid`, resolve the caller via `@CurrentUser`, delegate to a service, return DTOs.
Access rules come from `SecurityConfig` + `@PreAuthorize`.

`AuthController.java` `CONTROLLER` `/api/auth` — public

Method	Path	Description
POST	<code>/register</code>	Individual sign-up → JWT + 7-day trial
POST	<code>/register-institutional</code>	Sign-up with licence code → consumes a seat
POST	<code>/login</code>	Email + password → JWT + profile
POST	<code>/join-code</code>	Student links an existing account to an institution
GET	<code>/me</code>	Current profile + subscription

`SubscriptionController.java` `CONTROLLER` `/api/subscriptions`

Method	Path	Description
GET	<code>/me</code>	Current subscription (plan, status, days left)
POST	<code>/subscribe</code>	<code>{"plan": "MONTHLY" "YEARLY"}</code> — mock payment activation

`CourseController.java` `CONTROLLER` `/api/courses`

Method	Path	Description
GET	<code>/</code>	Subjects with questionCount + available years
GET	<code>/ {id}</code>	Course detail
GET	<code>/ {id}/years</code>	Year selection with per-year question counts

QuizController.java CONTROLLER /api/quizzes

Method	Path	Description
POST	/start	<code>{"courseId":1,"year":2023,"timed":true,"timeLimitSeconds":600}</code>
GET	/	My attempt history (paged)
GET	/ {id}	Attempt metadata (resume)
GET	/ {id}/questions/ {index}	One question — no correct answer
POST	/ {id}/answers	Save an answer + time spent
POST	/ {id}/submit	Grade & close (idempotent)
GET	/ {id}/result	Score + full review with explanations

DashboardController.java CONTROLLER /api/dashboard

GET /api/dashboard — greeting, daily quote, analytics widget and subscription in a single response.

ProgressController.java CONTROLLER /api/progress

Method	Path	Description
GET	/	Totals, per-course breakdown, daily trend, recent attempts
GET	/attempts	Paginated attempt history

InstitutionController.java

CONTROLLER**/api/institutions — INSTITUTION_ADMIN or ADMIN**

Method	Path	Description
POST	/	Admin: create institution + admin account
GET	/{{id}}	Overview incl. licence codes & seat usage
POST	/{{id}}/licenses	Generate a licence code
GET	/{{id}}/students	Students + progress snapshots
GET	/{{id}}/report/summary	Reporting dashboard data
GET	/{{id}}/report/export?format=csv pdf	Downloadable report

AdminController.java

CONTROLLER**/api/admin — ADMIN only**

Method	Path	Description
GET	/stats	Platform statistics
POST	/courses	Create course
POST	/questions	Add one question
POST	/questions/bulk	Upload a full paper (JSON array)
GET	/questions?courseId=&year=	List incl. correct answers
DELETE	/questions/{{id}}	Delete question
GET	/users?role=	List users
POST	/subscriptions/activate	Manually activate/extend a subscription
POST	/institutions	Create institution

12. Tests

QuizFlowIntegrationTest.java INTEGRATION

Boots the whole app on the H2 `demo` profile and exercises the API over real HTTP (MockMvc). Coverage:

- Full student journey: register → login → list courses → start quiz → fetch questions → answer → submit → result with review/explanation → progress → dashboard.
- Trial dates: `startDate = today, endDate = today + 7`.
- Institutional journey: onboarding with licence code → seat consumed → institution overview → student list → CSV & PDF exports.
- RBAC: student gets **403** on `/api/admin/**` ; bad password gets **401**; missing token gets **401**.

QuoteServiceTest.java UNIT

Verifies the daily quote rotation: same quote all day, changes on a different day (deterministic index), and a safe fallback when the table is empty.

Run: `./mvnw test` → 4 tests, 0 failures, 0 errors.

13. Demo Accounts & Seeded Data

Role	Email	Password	Notes
Platform admin	admin@quizora.app	Admin@123	Full admin panel
Student	student@demo.com	Student@123	Yearly subscription pre-activated
Institution admin	school@demo.com	School@123	"Demo JHS Accra"
Institution student	kofi@demo.com	Student@123	Onboarded via licence code

Licence code: `DEMO-CODE-1234` — 20 seats, valid 2026-01-01 → 2027-12-31.

Content: 6 JHS subjects, 44 BECE-style objective questions (2023 & 2024 papers), 12 motivational quotes.

Run it

```
# Zero setup (in-memory H2, auto-seeded)
./mvnw spring-boot:run -Dspring-boot.run.profiles=demo

# PostgreSQL
docker compose up -d
./mvnw spring-boot:run
```

14. API Reference Summary

Base URL `http://localhost:8080` · send `Authorization: Bearer <token>` unless marked public · errors are uniform JSON.

Group	Endpoints
Auth (public)	POST /api/auth/register · POST /api/auth/register-institutional · POST /api/auth/login · POST /api/auth/join-code · GET /api/auth/me
Subscriptions	GET /api/subscriptions/me · POST /api/subscriptions/subscribe
Courses	GET /api/courses · GET /api/courses/{id} · GET /api/courses/{id}/years
Quizzes	POST /api/quizzes/start · GET /api/quizzes · GET /api/quizzes/{id} · GET /api/quizzes/{id}/questions/{index} · POST /api/quizzes/{id}/answers · POST /api/quizzes/{id}/submit · GET /api/quizzes/{id}/result
Dashboard	GET /api/dashboard
Progress	GET /api/progress · GET /api/progress/attempts
Institutions	POST /api/institutions · GET /api/institutions/{id} · POST /api/institutions/{id}/licenses · GET /api/institutions/{id}/students · GET /api/institutions/{id}/report/summary · GET /api/institutions/{id}/report/export
Admin (ADMIN)	GET /api/admin/stats · POST /api/admin/courses · POST /api/admin/questions · POST /api/admin/questions/bulk · GET /api/admin/questions · DELETE /api/admin/questions/{id} · GET /api/admin/users · POST /api/admin/subscriptions/activate · POST /api/admin/institutions

Roadmap: Phase 2 — subjective questions (`QuestionSection.SUBJECTIVE` enum already exists). Phase 3 — WASSCE/SHS (`Level.SHS` exists). Payments — replace mock activation with Paystack/MoMo. Production — Flyway migrations, refresh tokens, rate limiting.